

CE 6320 Data Structures

Sumit Kumar - dal833771

Homework 2

1 Problem 1

Listing 1: **Problem1.h**

```
1  /* Huffman Coding */
2
3  #include <iostream>
4  using namespace std;
5
6  struct Node {
7      int weight;          // frequency or sum
8      char sym;            // 'A'....'Z' for leaves; 0 for
9                          // internal
10     Node *left, *right;
11     Node *next;
12 }
13
14 // Push node u at the front of the alive list
15 // Note: Passing a reference allows to change the value of
16 // the argument
17 void pushFront(Node* &head, Node* u);
18
19 // Find the two smallest-weight nodes in the alive list (
20 // single pass)
21 // Returns pointers to the minima and their predecessors (to
22 // splice in O(1))
23 void pickTwoMin(Node *head,
24                 Node* &min1, Node* &min1Prev,
25                 Node* &min2, Node* &min2Prev);
26
27 // Build the Huffman tree from the given dataset; return teh
28 // root
29 Node* buildHuffman();
30
31 // Pre-order traversal: print "<sym> : <bits>" at leaves
32 // buf holds the current path
```

```

28 // len: current length
29 // Use recursion
30 void emitCodesPreorder(Node *u, char buf[], int len);

```

Listing 2: Problem1.cpp

```

1 #include "Problem1.h"
2
3 // Push node u at the front of the alive list
4 // Note: Passing a reference allows to change the value of
   the argument
5 void pushFront(Node* &head, Node* u) {
6     u->next = head;
7     head = u;
8 }
9
10 // Find the two smallest-weight nodes in the alive list (
   single pass)
11 // Returns pointers to the minima and their predecessors (to
   splice in O(1))
12 void pickTwoMin(Node *head,
13                 Node* &min1, Node* &min1Prev,
14                 Node* &min2, Node* &min2Prev) {
15     Node *cur = head;
16     Node* prev = nullptr;
17
18     min1 = nullptr;
19     min1Prev = nullptr;
20     min2 = nullptr;
21     min2Prev = nullptr;
22
23     while (cur->next != nullptr)
24     {
25         if (min1 == nullptr || cur->weight < min1->weight) {
26             // Current min becomes second min
27             min2 = min1;
28             min2Prev = min1Prev;
29
30             // Current node becomes first min
31             min1 = cur;
32             min1Prev = prev;
33         }
34         else if (min2 == nullptr || cur->weight < min2->
35             weight) {
36             min2 = cur;
37             min2Prev = prev;
38         }
39     }
40     prev = cur;

```

```

40         cur = cur->next;
41     }
42 }
43
44 // Build the Huffman tree from the given dataset; return the
    root
45 Node* buildHuffman() {
46     Node *alive = nullptr;
47     // Create leaves A to F and push them into alive list (
        any order)
48     const char symbols[] = {'A', 'B', 'C', 'D', 'E', 'F'};
49     const int weights[] = {5, 9, 12, 13, 16, 45};
50
51     for (int i = 0; i < 6; ++i)
52     {
53         Node* u = new Node;
54         u->weight = weights[i];
55         u->sym = symbols[i];
56         u->left = nullptr;
57         u->right = nullptr;
58         u->next = nullptr;
59
60         pushFront(alive, u);
61     }
62     // Repeatedly pick two smallest, merge, and push parent
        back
63     while (alive != nullptr && alive->next != nullptr) {
64         Node *min1 = nullptr;
65         Node *min2 = nullptr;
66         Node *min1Prev = nullptr;
67         Node *min2Prev = nullptr;
68         pickTwoMin(alive, min1, min1Prev, min2, min2Prev);
69         // Remove min1 from list
70         if (min1Prev == nullptr) {
71             alive = min1->next;
72         }
73         else {
74             min1Prev->next = min1->next;
75         }
76
77         // Create parent node;
78         Node* parent = new Node;
79         parent->weight = min1->weight + min2->weight;
80         parent->sym = 0;
81         parent->left = min1;
82         parent->right = min2;
83         parent->next = nullptr;
84
85         // Put parent into alive list
86         pushFront(alive, parent);

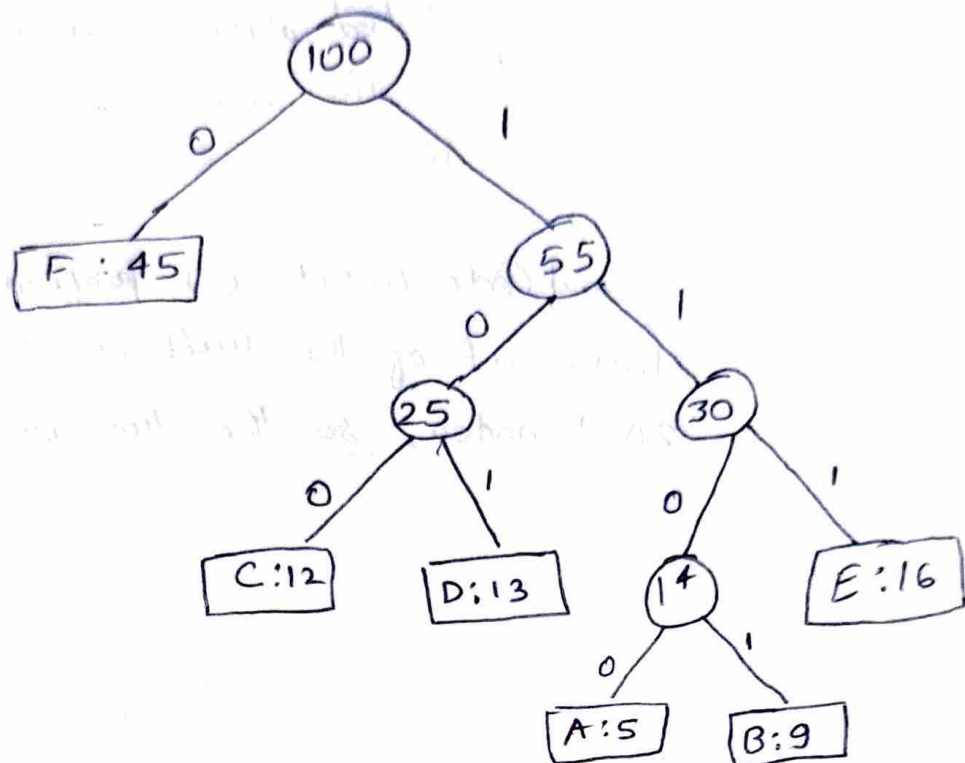
```

```

87     }
88
89     // Return the last remaining node
90     return alive;
91 }
92
93
94 // Pre-order traversal: print "<sym> : <bits>" at leaves
95 // buf holds the current path
96 // len: current length
97 // Use recursion
98 void emitCodesPreorder(Node *u, char buf[], int len) {
99     if (u == nullptr) return;
100
101     // Leaf
102     if (u->left == nullptr && u->right == nullptr) {
103         buf[len] = '\0';
104         cout << u->sym << "□:□" << buf << endl;
105         return;
106     }
107
108     // GO left = 0
109     buf[len] = '0';
110     emitCodesPreorder(u->left, buf, len+1);
111
112     // Go right = 1
113     buf[len] = '1';
114     emitCodesPreorder(u->right, buf, len+1);
115 }
116
117
118 int main() {
119     Node *root = buildHuffman();
120     char buf[64];
121     emitCodesPreorder(root, buf, 0);
122     return 0;
123 }

```

1.) Problem 1: Huffman Coding



3)

Step	Picked Weights	New Weight
1	5, 9	14
2	12, 13	25
3	14, 16	30
4	25, 30	55
5	45, 55	100

Final codes:

A : 1100

D : 101

B : 1101

E : 111

C : 100

F : 0

4. > Build Time: There are $N-1$ merge steps. At each step, the singly linked alive list is scanned once to find the smallest nodes. The list sizes are $N, N-1, N-2, \dots, 2$ giving total time $O(N^2)$.

Code Generation: `emitCodesPreorder()` performs 1 preorder traversal of the Huffman tree which contains $2N-1$ nodes, so the time is $O(N)$

2 Problem 2

Listing 3: Problem2.h

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     const char *name; // e.g., "R", "A1", "d1"
6     Node *left; // first downstream link (may be nullptr)
7     Node *right; // second downstream link (may be nullptr)
8 };
9
10 // Create a new node with given name.
11 Node* newNode(const char *label);
12
13 // Build the topology from the above description; return the
14 //    root ("R").
15 Node* buildNetwork();
16
17 // Return true iff u is an endpoint (leaf).
18 bool isLeaf(Node *u);
19
20 // Count the number of endpoints in the network, recursively
21 //    .
22 int countLeaves(Node *u);
23
24 // List endpoints using your chosen traversal (pick ONE
25 //    style and stick to it).
26 void listEndpoints(Node *u);
```

Listing 4: Problem2.cpp

```
1 #include "Problem2.h"
2
3
4 // Create a new node with given name.
5 Node* newNode(const char *label) {
6     Node* temp = new Node;
7     temp->name = label;
8     temp->left = nullptr;
9     temp->right = nullptr;
10    return temp;
11 }
12
13 // Build the topology from the above description; return the
14 //    root ("R").
15 Node* buildNetwork() {
```

```

15     // Create all nodes
16     Node* R = newNode("R");
17     Node* A1 = newNode("A1");
18     Node* A2 = newNode("A2");
19     Node* E1 = newNode("E1");
20     Node* E2 = newNode("E2");
21     Node* E3 = newNode("E3");
22     Node* d1 = newNode("d1");
23     Node* d2 = newNode("d2");
24     Node* d3 = newNode("d3");
25     Node* d4 = newNode("d4");
26     Node* d5 = newNode("d5");
27     Node* d6 = newNode("d6");
28     // Build tree
29     R->left = A1;
30     R->right = A2;
31     A1->left = E1;
32     A1->right = E2;
33     A2->left = E3;
34     A2->right = nullptr;
35
36     E1->left = d1;
37     E1->right = d2;
38     E2->left = d3;
39     E2->right = d4;
40     E3->left = d5;
41     E3->right = d6;
42
43     return R;
44 }
45
46 // Return true iff u is an endpoint (leaf).
47 bool isLeaf(Node *u) {
48     bool flag_leaf = (u != nullptr) && (u->right == nullptr)
49                     && (u->left == nullptr);
50     return flag_leaf;
51 }
52 // Count the number of endpoints in the network, recursively
53 .
54 int countLeaves(Node *u) {
55     if (u == nullptr) return 0;
56     if (isLeaf(u)) return 1;
57     int number_leaves = countLeaves(u->left) + countLeaves(u
58                             ->right);
59     return number_leaves;
60 }
61 // List endpoints using your chosen traversal (pick ONE
62     style and stick to it).

```



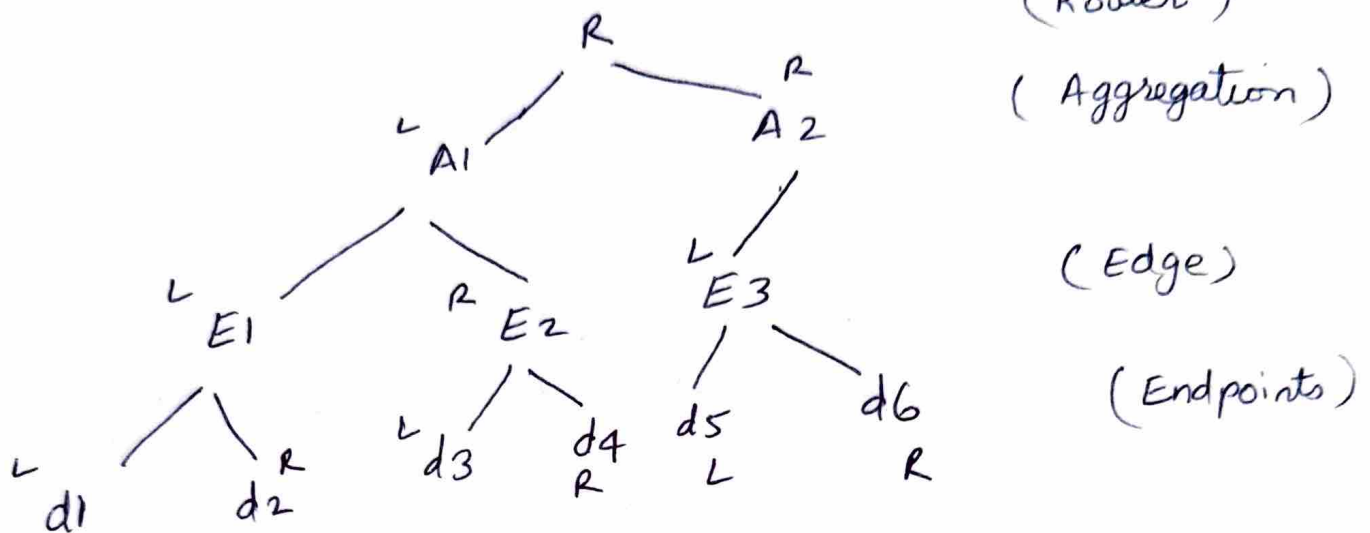
```

61 void listEndpoints(Node *u) {
62     if (u == nullptr) return;
63     if (isLeaf(u)) {
64         cout << u->name << " ";
65     }
66
67     // visit left subtree
68     listEndpoints(u->left);
69
70     // visit right subtree
71     listEndpoints(u->right);
72 }
73
74 int main() {
75     Node *root = buildNetwork();
76     std::cout << "Endpoints_(chosen_traversal):\n";
77     listEndpoints(root);
78     int leaves = countLeaves(root);
79     std::cout << "Total_endpoints:" << leaves << "\n";
80     return 0;
81 }

```

Problem 2 :

1.)



3.) List of Endpoints : d1, d2, d3, d4, d5, d6
Final Count = 6

4.) CountLeaves() visits every node exactly once. Every node gets processed once.
So Time complexity is $O(n)$

Auxiliary space: The function is recursive. At the deepest point, there can be approximately one recursive call for every level of the tree.

$\therefore O(h)$

3 Problem 3

Listing 5: Problem3.h

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     const char *name;
6     Node *left, *right, *parent;
7 };
8
9 // Create a new node with a given name.
10 Node* newNode(const char *s);
11
12 // Helper: set L and R as children of p and set their parent
13 // pointers.
14 void linkLR(Node *p, Node *L, Node *R);
15
16 // Build the given tree; return root.
17 Node* buildTree();
18
19 // Pre-order search by name; return pointer or nullptr.
20 Node* findPreorder(Node *u, const char *target);
21
22 // Depth in edges from u up to root (root has depth 0).
23 int depth(Node *u);
24
25 // LCA via parent pointers only.
26 // Steps: raise deeper node until both at same depth; then
27 // climb both until equal.
28 Node* lca(Node *a, Node *b);
29
30 // Compare two C-style strings
31 bool stringEqual(const char *a, const char *b);
```

Listing 6: Problem3.cpp

```
1 #include "Problem3.h"
2
3 // Create a new node with a given name.
4 Node* newNode(const char *s) {
5     Node* u = new Node;
6     u->name = s;
7     u->left = nullptr;
8     u->right = nullptr;
9     u->parent = nullptr;
10    return u;
11 }
```

```

11 }
12
13 // Helper: set L and R as children of p and set their parent
    pointers.
14 void linkLR(Node *p, Node *L, Node *R) {
15     p->left = L;
16     p->right = R;
17
18     if (L != nullptr)
19         L->parent = p;
20     if (R != nullptr)
21         R->parent = p;
22 }
23
24 // Build the given tree; return root.
25 Node* buildTree() {
26     Node* root = newNode("root");
27     Node* home = newNode("home");
28     Node* var = newNode("var");
29     Node* alice = newNode("alice");
30     Node* bob = newNode("bob");
31     Node* tmp = newNode("tmp");
32     Node* lib = newNode("lib");
33     Node* docs = newNode("docs");
34     Node* pics = newNode("pics");
35     Node* games = newNode("games");
36     Node* logs = newNode("logs");
37
38     linkLR(root, home, var);
39     linkLR(home, alice, bob);
40     linkLR(alice, docs, pics);
41     linkLR(bob, games, logs);
42     linkLR(var, tmp, lib);
43     return root;
44 }
45
46 // Pre-order search by name; return pointer or nullptr.
47 Node* findPreorder(Node *u, const char *target) {
48     if (u == nullptr) return nullptr;
49
50     // Check current node
51     if (stringEqual(u->name, target)) return u;
52
53     // Search left subtree
54     Node* result = findPreorder(u->left, target);
55
56     if (result != nullptr) return result;
57
58     // Search right subtree
59     return findPreorder(u->right, target);

```

```

60 }
61
62 // Compare two C-style strings
63 bool stringEqual(const char *a, const char *b) {
64     int i = 0;
65     while (a[i] != '\0' && b[i] != '\0') {
66         if (a[i] != b[i]) return false;
67         ++i;
68     }
69     return a[i] == b[i];
70 }
71
72 // Depth in edges from u up to root (root has depth 0).
73 int depth(Node *u) {
74     int d = 0;
75     while (u != nullptr && u->parent != nullptr) {
76         u = u->parent;
77         ++d;
78     }
79     return d;
80 }
81
82 // LCA via parent pointers only.
83 // Steps: raise deeper node until both at same depth; then
84 // climb both until equal.
85 Node* lca(Node *a, Node *b) {
86     int da = depth(a);
87     int db = depth(b);
88
89     // Move deeper node upward
90     while (da > db) {
91         a = a->parent;
92         --da;
93     }
94     while (db > da) {
95         b = b->parent;
96         --db;
97     }
98
99     // Now both nodes have same depth
100    // Move both up until they meet
101    while (a != b) {
102        a = a->parent;
103        b = b->parent;
104    }
105    return a;
106 }
107
108 int main() {
109     Node *root = buildTree();

```

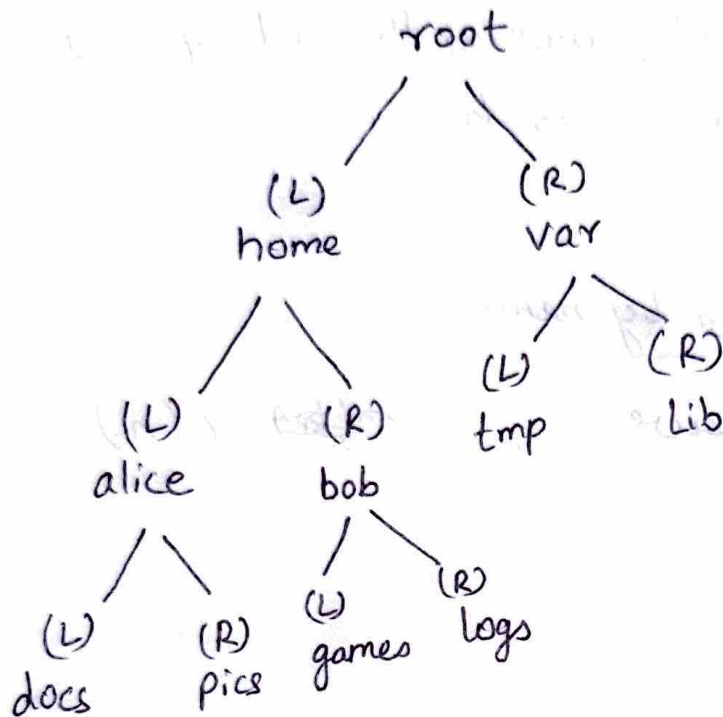
```

109     const char *Q[][2] = {
110         {"docs","pics"}, // expect: alice
111         {"docs","games"}, // expect: home
112         {"logs","tmp"}, // expect: root
113         {"lib","tmp"}, // expect: var
114         {"alice","games"} // expect: home
115     };
116     for (int i = 0; i < 5; ++i) {
117         Node *a = findPreorder(root, Q[i][0]);
118         Node *b = findPreorder(root, Q[i][1]);
119         Node *c = lca(a, b);
120         std::printf("LCA(%s,%s) = %s\n", Q[i][0], Q[i][1], c
121                     ? c->name : "(null)");
122     }
123     return 0;

```

Problem 3:

1.)



3.)

<u>Query</u>	<u>LCA</u>	<u>Pointer moves</u>
(docs, pics)	alice	raise both to alice
(docs, games)	home	raise docs to alice then home raise games to bob then home
(logs, tmp)	root	logs → bob → home → root tmp → var → root
(lib, tmp)	var	lib → var, tmp → var
(alice, games)	home	alice → home games → bob → home

4) Time Complexity of lca()

While calculating the depth, moving the node upward, maximum it can iterate is h .

So, $O(h)$

Time to locate a node by name :

Find Pre order () is recursive, ~~$O(h)$~~ $O(n)$

4 Problem 4

Listing 7: Problem4.h

```
1 #include <iostream>
2 #include <cstring> // for strcmp/strncmp
3 using namespace std;
4
5 struct Node {
6     const char *word; // stored as lowercase ASCII
7     Node *left, *right;
8 };
9
10 // Create a new node with a given C-string (assume lifetime
    //   >= program).
11 Node* newNode(const char *w);
12
13 // Insert word into BST by strcmp (ties to the right).
14 // Use recursive implementation.
15 // Return the (new or existing) node.
16 Node* insert(Node* &root, const char *w);
17
18 // Return true iff the word exists in the BST.
19 bool contains(Node *root, const char *w);
20
21 // Return the number of words that start with 'prefix'.
22 // Hint: recurse into both sides; count this node if strncmp
    //   (...)==0.
23 int prefixCount(Node *root, const char *prefix);
24
25 // In-order print (ascending)
26 void inorder(Node *u);
```

Listing 8: Problem4.cpp

```
1 #include "Problem4.h"
2
3 // Create a new node with a given C-string (assume lifetime
    //   >= program).
4 Node* newNode(const char *w) {
5     Node* u = new Node;
6     u->word = w;
7     u->left = nullptr;
8     u->right = nullptr;
9
10     return u;
11 }
12
```

```

13 // Insert word into BST by strcmp (ties to the right).
14 // Use recursive implementation.
15 // Return the (new or existing) node.
16 Node* insert(Node* &root, const char *w) {
17     Node* cur = nullptr;
18     if (root == nullptr) {
19         root = newNode(w);
20         return root;
21     }
22     if (strcmp(w, root->word) >= 0)
23     {
24         cur = insert(root->right, w);
25     }
26     else {
27         cur = insert(root->left, w);
28     }
29     return cur;
30 }
31
32 // Return true iff the word exists in the BST.
33 bool contains(Node *root, const char *w) {
34     bool node_present = false;
35     Node* cur = root;
36     if (root == nullptr) {
37         return false;
38     }
39     int cmp = strcmp(w, root->word);
40
41     if (cmp == 0) return true;
42     if (cmp < 0) return contains(root->left, w);
43     return contains(root->right, w);
44 }
45
46 // Return the number of words that start with 'prefix'.
47 // Hint: recurse into both sides; count this node if strncmp
48 // (...)==0.
49 int prefixCount(Node *root, const char *prefix) {
50     if (root == nullptr) return 0;
51     int count = 0;
52
53     if (strncmp(root->word, prefix, strlen(prefix)) == 0)
54         count = 1;
55
56     count += prefixCount(root->left, prefix);
57     count += prefixCount(root->right, prefix);
58     return count;
59 }
60
61 // In-order print (ascending)
62 void inorder(Node *u) {

```

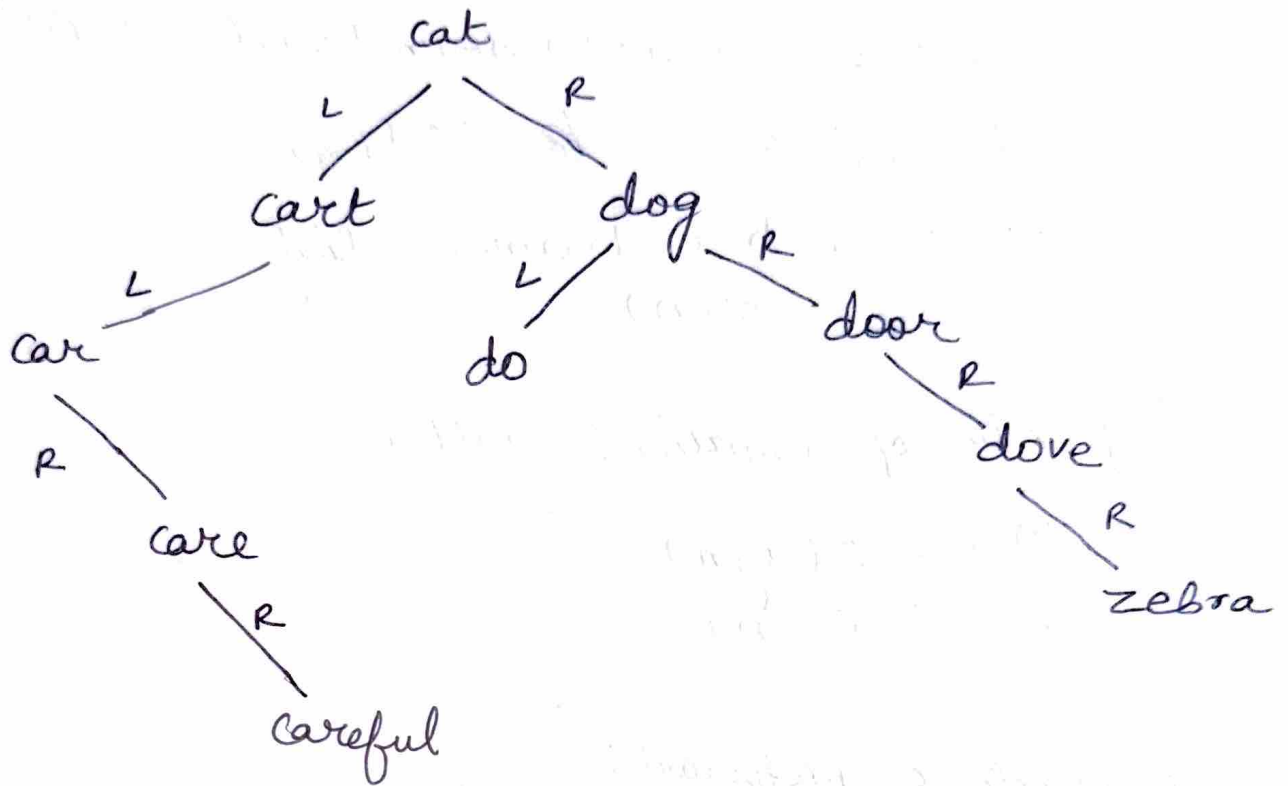
```

61     if (u == nullptr) return;
62     inorder(u->left);
63     cout << u->word << " ";
64     inorder(u->right);
65 }
66
67
68
69 int main() {
70     Node *root = nullptr;
71     // Insert fixed dataset (one call per word):
72     insert(root, "cat");
73     insert(root, "dog");
74     insert(root, "door");
75     insert(root, "dove");
76     insert(root, "cart");
77     insert(root, "car");
78     insert(root, "care");
79     insert(root, "careful");
80     insert(root, "do");
81     insert(root, "zebra");
82     // Example checks (optional):
83     std::cout << "contains('care')_=" << contains(root, "
        care") << std::endl;
84     std::cout << "prefixCount('car')_=" << prefixCount(root
        , "car") << std::endl;
85     return 0;
86 }

```

Problem 4 :

1.)



3) * Contains ("care") returns true.

$cat \xrightarrow{L} cart \xrightarrow{L} car \xrightarrow{R} care$

* prefix Count ("do") = 4

prefix Count ("car") = 4

prefixCount ("z") = 1

4.) * Complexity of insert () :

Each recursive call moves down 1 level : $O(h)$

For balanced BST : ~~to~~ $O(\log n)$

Worst case : tree becomes a list
 $O(n)$

* Complexity of contains : $O(h)$

Average : $O(\log n)$

Worst : $O(n)$

* Complexity of prefix Count :

As we recurse both sides and visit every node to compare the strings.. Therefore in the worst case,

$O(n)$